Northstar Integration Specs

Oct 22, 2026 · @Abhishek

1. Common adapter pattern

Every external service sits behind one Java interface in the `integrations` module, with a real adapter and a fake adapter for tests; nothing outside that module calls a provider directly.

Inbound (webhooks)

1. Endpoint per provider: `POST /webhooks/{provider}`.
2. Verify signature first; invalid → 401, stored with `signature_valid = false`.
3. Insert into `webhook_events` (unique on provider event ID); duplicate → 200 and stop.
4. Resolve tenant (from phone number ID, account ID or reference ID), queue a job, return 200 within a few hundred milliseconds.
5. The worker normalises the payload into an engine event (`events` table) and processes it.

Outbound (actions)

- Every call is an `actions` row executed by the worker with an idempotency key; provider idempotency headers used where offered.
- Provider errors are classified: **retryable** (timeouts, 429, 5xx) → backoff; **permanent** (invalid input, revoked auth) → Failed + alert; **auth** (expired token) → integration status error + pause dependent actions.
- Rate limits respected per tenant and per provider with a token bucket in the worker.

Health

- `integrations.status`, `last_success_at`, `last_error` updated on every call.
- Dashboard shows connected / error / reconnect per integration; Northstar admin sees all tenants.

Logging: request IDs and status codes only; never message bodies, tokens or card data.

2. WhatsApp Cloud API

Northstar operates as a Meta Tech Provider: each business connects its own WhatsApp number through Embedded Signup, and one Northstar app receives every tenant's webhooks.

Onboarding (per business)

1. Business runs Embedded Signup from the onboarding wizard; Northstar receives the WABA ID, phone number ID and a business token.
2. Store in `whatsapp_accounts` and `integration_credentials`; subscribe the WABA to Northstar's app webhooks.
3. Create the starter message templates in the business's WABA; track approval status in `message_templates`.
4. Send a test message; readiness check passes when inbound and outbound both work.

App permissions to request: `whatsapp_business_management`, `whatsapp_business_messaging`, and `whatsapp_business_manage_events` (needed for the Conversions API, section 5).

Webhooks

Item	Rule
Verification	`GET` with `hub.mode`, `hub.verify_token`, `hub.challenge`; echo the challenge when the token matches
Signature	`X-Hub-Signature-256`: HMAC-SHA256 of the raw body with the app secret; reject on mismatch
Routing	`metadata.phone_number_id` → `whatsapp_accounts` → tenant
Dedupe	Message ID for messages; message ID + status for status updates
Inbound types handled	text, interactive button reply, image, document, audio, location
Status updates	sent, delivered, read, failed → `messages.status`, failures → action error

Item	Rule
Click-to-WhatsApp referral	`referral` object on the first message: `source_id`, `source_type`, `source_url`, `headline`, `ctwa_clid` → `ad_referrals`; `ctwa_clid` stored for Conversions API matching

Sending

- `POST /{phone-number-id}/messages` on the Graph API with the business token.
- Free-form text and media only inside the 24-hour customer-service window (`conversations.window_expires_at`); outside it, an approved template.
- Quote and invoice PDFs sent as `document` messages with a signed URL or uploaded media ID.
- Accept / Request Changes as interactive reply buttons (button IDs carry the quote version ID).
- Opt-out keywords (STOP and tenant-configured words) checked on every inbound message before the AI sees it.

Quality and limits: store the number's quality rating and messaging limit; alert the admin when quality drops.

3. Razorpay

Each business collects into its own Razorpay account; Northstar creates payment links on its behalf and marks payments paid only from Razorpay's signed webhooks, backed by a reconciliation job.

Connection: the business connects its account in onboarding (API key pair or Razorpay partner authorisation, open decision); Northstar registers a webhook on that account with a per-tenant webhook secret.

Creating a payment link

- Payment Links API with: amount in paise, currency INR, customer name and phone, description (quote number + kind), `reference_id` = Northstar payment ID, `expire_by` = link expiry, notifications off (Northstar sends the link on WhatsApp).
- Store `razorpay_link_id`, link URL and expiry on the `payments` row.

Webhook events to subscribe

Event	Northstar action
`payment_link.paid`	Payment → paid; engine event `payment.captured`; lead moves on
`payment_link.partially_paid`	Payment → partial; remainder link flow (Edge case W19)
`payment_link.expired`	Payment → expired; slot released; fresh-link flow
`payment_link.cancelled`	Payment → expired (cancelled by staff)
`payment.failed`	Record failure reason; fresh link after 1 failure, alert after 2
`refund.processed`	Refund → processed; ledger + credit note finalised
`refund.failed`	Refund → failed; owner alerted

Validation: verify the signature header (`X-Razorpay-Signature`, HMAC-SHA256 of the raw body with the tenant's webhook secret) before anything else; dedupe on the Razorpay event ID header and on `razorpay_payment_id`. Confirm header names against Razorpay's webhook validation docs during build.

Refunds: created through the Refunds API only after owner approval; amount from the cancellation policy; refund ID stored on `refunds`.

Reconciliation: every 15 minutes, a job fetches the status of payment links still pending past their expected time and of refunds still processing, so a delayed or missed webhook never leaves a booking stuck.

Never: store card or UPI details, mark paid from a client's screenshot or message, or let the AI change payment status.

4. Google: OAuth, Calendar and Sheets

One Google Cloud project serves all tenants; asking for the narrowest scopes keeps Google's app verification simpler and protects staff privacy.

OAuth

- Authorization-code flow with offline access; refresh tokens encrypted in `integration_credentials`.

- Business admin connects the business Google account (business calendar + Sheets); each staff member connects their own calendar.
- Scopes, narrowest that works: Calendar events read/write + free/busy for calendars; `drive.file` for Sheets, so Northstar can only touch spreadsheets it created or the user picked. Confirm each scope's sensitivity class in Google's scope list; Calendar scopes need app verification before outside users can connect.
- Revoked or expired refresh token → integration error, dependent actions pause, reconnect prompt.

Calendar

Need	How
Availability	Free/busy query for the candidate staff calendars and the business calendar over the requested window; returns busy times only, so private event details are never read
Cache	`busy_blocks` refreshed on query and by push notifications (watch channels) on each connected calendar, plus a periodic resync
Create	On Scheduled: event in business calendar and the assigned staff calendar; IDs saved in `appointments.google_event_ids`
Update	Reschedule or reassignment patches the event, or deletes from the old staff calendar and creates in the new one
Cancel	Delete events; platform appointment kept as Cancelled
Conflicts	A new Google event overlapping a job → conflict flag to admin (Edge case W29)

Sheets

- Northstar creates the spreadsheet (two tabs: Leads, Payments) in the business's Drive during onboarding, or the admin picks one through the Google Picker.
- Header row written by Northstar; first column `platform_record_id`, last columns `updated_at` and `sync_status`.

- Writes come from `sync_outbox` in batches per tenant (values batch update / append); rows located by `platform_record_id`, never by row number.
- Quotas are per project and shared by all tenants: batch writes, back off on 429, and request a quota increase before about 100 tenants.
- Inbound edits: only columns listed in `sheet_mappings.inbound_fields`, read on a schedule; compared against `updated_at`; protected-field edits are reverted and logged to `sheet_conflicts`.
- Never written to sheets: message transcripts, tokens, card or bank data.

5. Meta Conversions API and Marketing API

Lead stages go back to Meta through the Conversions API for business messaging, matched on the click ID from each click-to-WhatsApp chat; Meta does not deduplicate these events, so Northstar must send each one exactly once.

Setup (per business)

- Requires app permissions `whatsapp_business_management` and `whatsapp_business_manage_events`, Marketing API access, and the business connected through Embedded Signup (Meta guide).
- Create or fetch the dataset for the business's WABA (`POST/GET /{WABA_ID}/dataset`); store the dataset ID. Only one dataset can be linked to a page.

Sending an event: `POST /{DATASET_ID}/events` on the Graph API with `data[]` and `partner_agent` = "northstar".

Field	Value
`event_name`	From the mapping below
`event_time`	Unix time of the stage change
`action_source`	`business_messaging`
`messaging_channel`	`whatsapp`
`user_data`	`whatsapp_business_account_id` + `ctwa_clid` (preferred); without a click ID: `page_id` + WABA ID + SHA-256 hashed phone

Field	Value
`custom_data`	`currency` = INR, `value` in rupees (Purchase and InitiateCheckout)

Stage → event mapping (configurable per business)

Northstar stage	Meta event	Role
New lead from an ad	`LeadSubmitted`	Volume signal
Scored Warm or Hot	`QualifiedLead`	Quality signal
Payment pending (pre-final, default)	`InitiateCheckout`	Main optimisation event
Completed	`Purchase` with value	Value signal for ROAS

Rules

- One event per lead per event name: `meta_events` unique on (tenant, lead, event_name); Northstar never resends a succeeded event.
- Leads without a click ID (chat links, direct) are not sent by default; matching on hashed phone is limited and not guaranteed.
- Only events that happened in the WhatsApp thread go through this API.
- Each business can check events in its Events Manager; the onboarding guide shows them where.

Marketing API (inbound): with `ads_read` on the business's ad account, a daily job pulls campaign, ad set and ad names plus daily spend into `ad_spend_daily`, enabling cost per lead, per qualified lead and per booking.

6. Chat links, email, LLM and PDF services

These four are smaller integrations, but each follows the same adapter, retry and logging rules as section 1.

Tracked chat links

1. `GET northstar.com/c/{code}` → look up `chat_links`; unknown or inactive → friendly error page.

2. Create a `link_clicks` row with a short unique `ref_code`, UTM parameters and page referrer.
3. Redirect (302) to `https://wa.me/{business_number}?text=<prefill text + ref_code>`.
4. When the inbound message contains a known `ref_code`, attach the click and link to the lead; tag source.
5. QR codes generated with ZXing as PNG and SVG, stored in Supabase Storage.

Email (Resend)

- Sending domain verified (SPF, DKIM) before launch; separate sub-domain for transactional mail.
- Used for: staff invites, email verification, approval and failure notifications, Northstar's own subscription invoices.
- Bounces and complaints recorded; repeated bounces mark the address invalid.

LLM provider

- Accessed through Spring AI behind Northstar's own `AiService` interface, so the provider can change without touching the engine.
- Each call sends: system prompt (versioned), tenant's approved FAQ and catalogue excerpt, the lead's current fields, the last N messages, and only the tools allowed for the lead's stage.
- Structured output validated against a schema; invalid output → one retry, then Needs Human.
- Per-tenant token budgets and rate limits; usage written to `usage_daily`.
- Provider data-retention and training settings configured to the most restrictive option offered.

PDF service (Gotenberg)

- Internal-only container; reachable from the worker, never from the internet.
- Flow: fill the tenant's Word template with poi-tl → send the .docx to Gotenberg → receive PDF → store in Supabase Storage under `tenant_id/invoices/` → send on WhatsApp.
- Timeout 30 seconds per document; failure → action retry (Edge case W22).

7. Secrets, environments and test accounts

Staging and production never share credentials, and staging uses only provider test modes and test numbers.

Integration	Staging	Production
WhatsApp	Meta test number on a separate test app	Northstar's live app; each business's own number
Razorpay	Test mode keys; test card and UPI flows	Each business's live account
Google	Separate OAuth client; test users only	Verified OAuth client
Meta CAPI	Test events code in Events Manager	Live dataset per business
LLM	Separate API key with a low budget	Production key with per-tenant budgets
Email	Sandbox or test domain	Verified sending domain

Platform secrets (Northstar's own app secrets, signing keys, database roles) live in DigitalOcean App Platform encrypted environment variables, never in the repo. **Tenant secrets** (business tokens, refresh tokens, Razorpay keys) live encrypted in `integration_credentials`, decrypted only inside the worker at call time. Rotation: platform secrets every 90 days or on staff change; tenant tokens refreshed per provider rules.

8. Open questions

- [ ] Razorpay connection model: business API keys, or Razorpay partner authorisation?
- [ ] Who pays WhatsApp message charges: business's own Meta payment method (default assumption) or Northstar?
- [ ] Should leads without a click ID be sent to Meta using hashed phone, given matching is limited?
- [ ] Google Picker for existing sheets, or always create a new Northstar sheet?
- [ ] First LLM provider and model, and per-tenant monthly token budget

Sources

- Meta: Conversions API for Business Messaging onboarding guide
- Razorpay: Payment Links webhooks
- Razorpay: all webhook events

WhatsApp Cloud API and Google API details are from general platform knowledge; confirm endpoints, scopes and quotas against current provider docs during build.